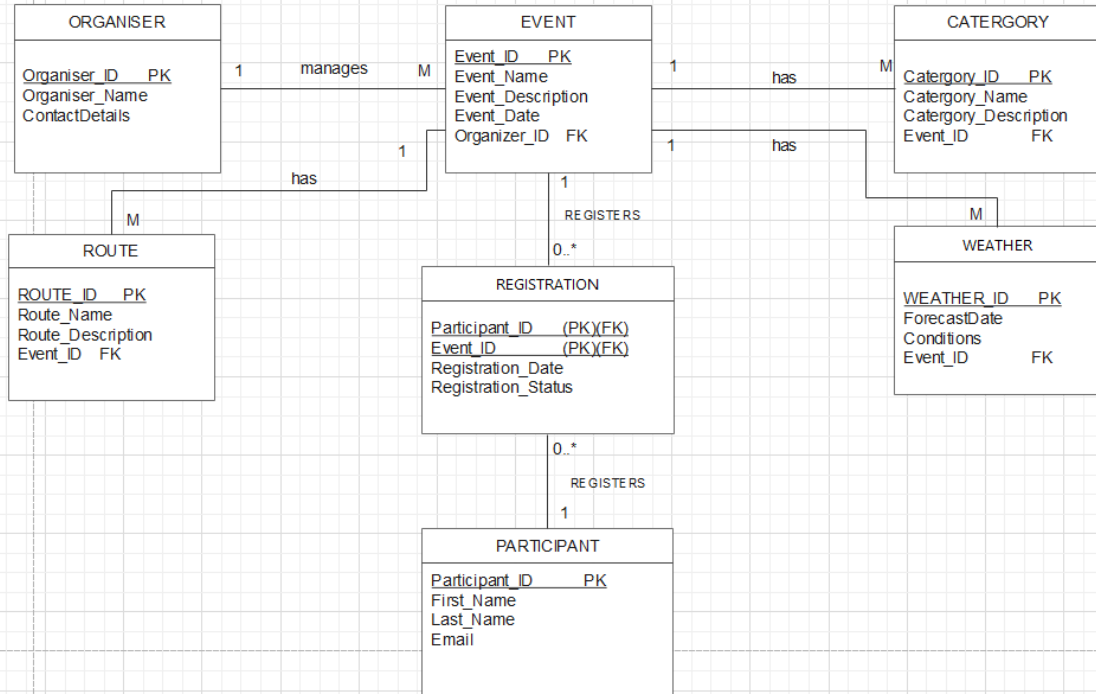


ST10476405

SANDILE SIYO

POE PART 1

SANDILE'S RACEDAY ERD



HTT P Met hod	Route	Descripti on	Role Requir ed	Request Body	Expected Response
POST	/api/auth/register	Registers a new user/participant account on the RaceDay system.	None	{ firstName, lastName, email, password }	201 Created – account created. 400 Bad Request – invalid data. 409 Conflict – email already exists.
POST	/api/auth/login	Authenticates a registered user and provides an access token.	None	{ email, password }	200 OK – login successful and token returned. 401 Unauthorized – invalid credentials.
GET	/api/users/me	Retrieves the profile information of the currently logged-in participant.	Participant	None	200 OK – participant profile returned. 401 Unauthorized – user is not logged in.
PUT	/api/users/me	Updates the profile information of the currently logged-in	Participant	{ firstName, lastName, email, phone, dateOfBirth }	200 OK – profile updated. 400 Bad Request – invalid data. 401

		participan t.			Unauthori zed – not logged in.
GET	/api/events	Retrieves a list of upcoming RaceDay events.	None	None	200 OK – list of events returned.
GET	/api/events/{id}	Retrieves detailed informatio n about a specific event.	None	None	200 OK – event details returned. 404 Not Found – event does not exist.
POST	/api/events	Creates a new RaceDay event.	Organi ser	{ eventName, eventDescripti on, eventDate, organiserId }	201 Created – event created. 400 Bad Request – invalid data. 401 Unauthori zed – not logged in.
PUT	/api/events/{id}	Updates an existing event.	Organi ser	{ eventName, eventDescripti on, eventDate }	200 OK – event updated. 404 Not Found – event does not exist.
DELETE	/api/events/{id}	Removes an event from the system.	Organi ser	None	204 No Content – event deleted. 404 Not Found – event does not exist.

GET	/api/events/{eventId}/categories	Retrieves all categories available for a specific event.	None	None	200 OK – categories returned. 404 Not Found – event does not exist.
POST	/api/events/{eventId}/categories	Creates a new category for a specific event.	Organiser	{ categoryName, distanceKM, entryFee, ageGroup }	201 Created – category created. 400 Bad Request – invalid data.
PUT	/api/categories/{id}	Updates an existing event category.	Organiser	{ categoryName, distanceKM, entryFee, ageGroup }	200 OK – category updated. 404 Not Found – category does not exist.
DELETE	/api/categories/{id}	Deletes an event category.	Organiser	None	204 No Content – category deleted. 404 Not Found – category does not exist.
POST	/api/events/{eventId}/enrolments	Registers the logged-in participant for a specific event.	Participant	{ registrationStatus }	201 Created – enrolment created. 404 Not Found – event does not exist. 409 Conflict –

					participant is already registered.
GET	/api/users/me/enrolments	Retrieves all events that the logged-in participant has registered for.	Participant	None	200 OK – participant's enrolments returned. 401 Unauthorized – not logged in.
GET	/api/events/{eventId}/enrolments	Retrieves all participants enrolled in a specific event.	Organiser	None	200 OK – list of enrolled participants returned. 404 Not Found – event does not exist.
DELETE	/api/events/{eventId}/enrolments	Cancels the logged-in participant's enrolment in an event.	Participant	None	204 No Content – enrolment cancelled. 404 Not Found – enrolment does not exist.
GET	/api/events/{eventId}/results	Retrieves the results for all participants in a specific event.	None	None	200 OK – event results returned. 404 Not Found – event does not exist.
GET	/api/users/me/results	Retrieves the logged-in participant's	Participant	None	200 OK – participant results returned. 401

		personal performance history.			Unauthorized – not logged in.
GET	/api/results/{id}	Retrieves the details of a specific race result.	None	None	200 OK – result returned. 404 Not Found – result does not exist.
POST	/api/events/{eventId}/results	Records a participant's result for a completed event.	Organiser	{ participantId, categoryId, finishTime, overallPosition, categoryPosition, pace, resultStatus }	201 Created – result recorded. 400 Bad Request – invalid data. 404 Not Found – event/participant does not exist.
PUT	/api/results/{id}	Updates a participant's race result.	Organiser	{ finishTime, overallPosition, categoryPosition, pace, resultStatus }	200 OK – result updated. 404 Not Found – result does not exist.
DELETE	/api/results/{id}	Removes an incorrect result from the system.	Organiser	None	204 No Content – result deleted. 404 Not Found – result does not exist.

HTT P Met hod	Route	Descripti on	Role Requir ed	Request Body	Expected Response
GET	/api/events/{eventId}/weather	Retrieves the weather forecast associated with a specific event.	None	None	200 OK – weather information returned. 404 Not Found – event/weather information does not exist.
POST	/api/events/{eventId}/weather	Adds or updates weather forecast information for an event.	Organiser	{ forecastDate , conditions }	201 Created – weather information saved. 400 Bad Request – invalid data.

Date, C.J. (2004)

SQL

part 1.sql -...Student (54)) ✕

```
1  create database RACEDAY;
2  USE RACEDAY;
3
4  CREATE TABLE Organisers(
5  Organiser_id INT PRIMARY KEY IDENTITY(1,1),
6  OrganiserName VARCHAR(100) NOT NULL,
7  OrganiserDetails VARCHAR(150) NOT NULL
8  );
9
10 CREATE TABLE EVENTS(
11 Event_ID int PRIMARY KEY IDENTITY (1,1),
12 Event_Name varchar(50) NOT NULL,
13 Event_Description varchar(250),
14 Event_Date Date,
15 Organiser_id int,
16 foreign key (Organiser_id) references Organiser(Organiser_id)
17 );
18
19 CREATE TABLE Routes (
20 Route_ID INT PRIMARY KEY IDENTITY (1,1),
21 RouteName VARCHAR(100) NOT NULL,
22 RouteDescription VARCHAR(200),
23 Event_ID int,
24 FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID)
25 );
26
27 CREATE TABLE Weathers (
28 WeatherID INT PRIMARY KEY IDENTITY (1,1),
29 Forecast_Date date,
30 Conditions VARCHAR(100),
31 Event_ID int,
32 FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID)
33 );
34
35 CREATE TABLE Participants (
36 Participant_ID INT PRIMARY KEY IDENTITY(1,1),
37 First_Name VARCHAR(100) NOT NULL,
38 Last_Name VARCHAR(100) NOT NULL,
39 DateRegistered DATE NOT NULL
40 );
```

part 1.sql -...Student (54)

```
42 CREATE TABLE Registrations (  
43 RegistrationID INT PRIMARY KEY IDENTITY(1,1),  
44 Registration_Status VARCHAR(50),  
45 Registration_Date DATE NOT NULL,  
46 Event_ID INT,  
47 Participant_ID INT,  
48 FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID),  
49 FOREIGN KEY (Participant_ID) REFERENCES Participant(Participant_ID)  
50 );  
51  
52  
53 -- Insert organisers  
54 INSERT INTO Organiser (OrganiserName, OrganiserDetails)  
55 VALUES ('RaceDay Events SA', 'Professional event management company'),  
56         ('ActiveLife Sports', 'Organises cycling and running events');  
57  
58 -- Insert events  
59 INSERT INTO Event (Event_Name, Event_Description, Event_Date, Organiser_id)  
60 VALUES ('Jozi Marathon', 'Annual city marathon through Johannesburg', '2026-10-15', 1),  
61         ('Cape Cycle Challenge', 'Road cycling event around Cape Town', '2026-11-05', 2);  
62  
63 -- Insert routes  
64 INSERT INTO Route (RouteName, RouteDescription, Event_ID)  
65 VALUES ('City Loop', 'Urban route covering 42km', 1),  
66         ('Coastal Ride', 'Scenic coastal route of 80km', 2);  
67  
68 -- Insert weather forecasts  
69 INSERT INTO Weather (Forecast_Date, Conditions, Event_ID)  
70 VALUES ('2026-10-15', 'Sunny with mild wind', 1),  
71         ('2026-11-05', 'Partly cloudy with light rain', 2);  
72  
73 -- Insert participants  
74 INSERT INTO Participant (First_Name, Last_Name, DateRegistered)  
75 VALUES ('Thabo', 'Mokoena', '2026-08-01'),  
76         ('Lerato', 'Dlamini', '2026-08-03'),  
77         ('Sipho', 'Nkosi', '2026-08-05');  
78  
79 -- Insert registrations  
80 INSERT INTO Registration (Registration_Status, Registration_Date, Event_ID, Participant_ID)  
81 VALUES ('Confirmed', '2026-08-10', 1, 1),
```

```

-- Insert participants
INSERT INTO Participant (First_Name, Last_Name, DateRegistered)
VALUES ('Thabo', 'Mokoena', '2026-08-01'),
       ('Lerato', 'Dlamini', '2026-08-03'),
       ('Sipho', 'Nkosi', '2026-08-05');

-- Insert registrations
INSERT INTO Registration (Registration_Status, Registration_Date, Event_ID, Participant_ID)
VALUES ('Confirmed', '2026-08-10', 1, 1),
       ('Pending', '2026-08-12', 1, 2),
       ('Confirmed', '2026-08-15', 2, 3);

select * from Organisier;
select * from Weather;
select * from Participant;
select * from Registration;
select * from Route;
select * from Event;

```

REFERENCE LIST

- **Connolly, T. & Begg, C.** (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th ed. Harlow: Pearson Education.
- **Elmasri, R. & Navathe, S.B.** (2016) *Fundamentals of Database Systems*. 7th ed. Boston: Pearson.
- **Date, C.J.** (2004) *An Introduction to Database Systems*. 8th ed. Boston: Addison-Wesley.
- **Fowler, M.** (2003) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd ed. Boston: Addison-Wesley.
- **Fielding, R.T.** (2000) *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
- **ISO/IEC** (2011) *Information technology — Database languages — SQL — Part 1: Framework (SQL/Framework)*. International Organization for Standardization.
- **Microsoft Docs** (2026) *SQL Server Documentation*. Available at: <https://learn.microsoft.com/sql/> (Accessed: 4 September 2026).
- **REST API Tutorial** (2026) *REST API Design Best Practices*. Available at: <https://restfulapi.net/> (Accessed: 4 September 2026).

